Дуальный код Хэмминга как симплексный код

Постановка задачи

Необходимо проверить и обосновать, что код, дуальный коду Хэмминга, действительно является симплексным.

---

1. Обозначения

Пусть задан двоичный код Хэмминга с параметром $r \geq 2$.

Его проверочная матрица H_r имеет размер

$$r \times (2^r - 1)$$

и состоит из всех ненулевых двоичных векторов длины r в качестве столбцов.

Следовательно,

$$n = 2^r - 1.$$

Сам код Хэмминга задаётся как

$$C_H = \{x \in \mathrm{GF}(2)^n \mid H_r x^T = 0\}.$$

Тогда дуальный код имеет вид

$$C_H^\perp = \mathrm{RowSpace}(H_r).$$

То есть строки матрицы H_r образуют порождающую матрицу дуального кода.

---

2. Параметры дуального кода

Поскольку среди столбцов матрицы H_r содержатся стандартные базисные векторы

$$e_1, e_2, \ldots, e_r,$$

её ранг равен r:

$$\mathrm{rank}(H_r) = r.$$

Следовательно,

$$\dim C_H^\perp = r.$$

Значит дуальный код к коду Хэмминга имеет параметры

$$[n, k] = [2^r - 1, r].$$

---

3. Почему все ненулевые слова имеют одинаковый вес

Пусть

$$m \in \mathrm{GF}(2)^r, \qquad m \neq 0.$$

Соответствующее кодовое слово дуального кода равно

$$c = mH_r.$$

Если v_j обозначает j-й столбец матрицы H_r, то j-я координата кодового слова равна

$$c_j = \langle m, v_j \rangle,$$

где скалярное произведение берётся над полем $\mathrm{GF}(2)$.

Рассмотрим линейный функционал

$$f_m(v) = \langle m, v \rangle.$$

Так как $m \neq 0$, этот функционал не является нулевым. Поэтому его ядро имеет размерность $r - 1$, а значит

$$|\ker f_m| = 2^{r-1}.$$

Во всём пространстве $\mathrm{GF}(2)^r$ ровно 2^{r-1} векторов дают значение 0 и ровно 2^{r-1} векторов дают значение 1.

Нулевой вектор всегда принадлежит ядру, то есть он даёт значение 0. Следовательно, среди всех **ненулевых** векторов пространства ровно

$$2^{r-1}$$

дают значение 1.

Но столбцы матрицы H_r и есть все ненулевые двоичные векторы длины r. Поэтому в кодовом слове c ровно 2^{r-1} единиц, то есть

$$w(c) = 2^{r-1}.$$

Итак, для любого ненулевого кодового слова дуального кода выполнено

$$w(c) = 2^{r-1}.$$

---

4. Почему расстояния между любыми двумя различными словами одинаковы

Пусть $c_1, c_2 \in C_H^{\perp}$, причём $c_1 \neq c_2$.

Так как код линейный,

$$c_1 + c_2 \in C_H^{\perp}.$$

Кроме того, из $c_1 \neq c_2$ следует

$$c_1 + c_2 \neq 0.$$

Для двоичных слов расстояние Хэмминга выражается через вес так:

$$d(c_1, c_2) = w(c_1 + c_2).$$

Но любое ненулевое кодовое слово дуального кода имеет вес 2^{r-1}. Следовательно,

$$d(c_1, c_2) = 2^{r-1}$$

для любых различных c_1 и c_2.

Таким образом, дуальный код коду Хэмминга действительно является симплексным кодом.

---

5. Итоговый результат

Дуальный код к двоичному коду Хэмминга имеет параметры

$$[2^r - 1, r, 2^{r-1}],$$

и обладает следующими свойствами:

1. Все ненулевые кодовые слова имеют одинаковый вес 2^{r-1}.
2. Расстояние между любыми двумя различными кодовыми словами одинаково и равно 2^{r-1}.
3. Число ненулевых кодовых слов равно

$$2^r - 1.$$

Следовательно, этот код является симплексным.

---

6. Что делает приложенная программа на C++

Приложенная программа:

1. Строит проверочную матрицу H_r кода Хэмминга.
2. Использует её как порождающую матрицу дуального кода.
3. Генерирует все кодовые слова дуального кода.
4. Проверяет, что все ненулевые слова имеют одинаковый вес.
5. Проверяет, что все попарные расстояния одинаковы.
6. Проверяет, что все попарные BPSK-корреляции одинаковы.
7. Проверяет ранг матрицы и корректность её столбцов.

Код программы:

```
#include <algorithm>
#include <cassert>
#include <cstdint>
#include <iostream>
#include <numeric>
#include <set>
#include <string>
#include <vector>

using Bit = std::uint8_t;
using Word = std::vector<Bit>;
using Matrix = std::vector<Word>;

struct VerificationReport
{
    int r{};
    int n{};
    int k{};
    int expected_weight{};
    int nonzero_codeword_count{};

    bool generator_has_full_row_rank{};
    bool all_columns_nonzero_and_unique{};
    bool all_nonzero_weights_equal{};
    bool all_pairwise_distances_equal{};
    bool all_bpsk_correlations_equal{};

    std::set<int> observed_weights{};
    std::set<int> observed_distances{};
    std::set<int> observed_bpsk_correlations{};
};
```

```
auto row_count(const Matrix& matrix) -> int
{
    return static_cast<int>(matrix.size());
}

auto column_count(const Matrix& matrix) -> int
{
    assert((!matrix.empty()) && "matrix must not be empty");
    return static_cast<int>(matrix.front().size());
}

auto to_bit_string(const Word& word) -> std::string
{
    std::string s;
    s.reserve(word.size());

    for (const Bit bit : word)
    {
        s.push_back(bit == 0 ? '0' : '1');
    }

    return s;
}

auto build_hamming_parity_check_matrix(const int r) -> Matrix
{
    assert((r >= 2) && "r must be at least 2");
    assert((r < 20) && "exhaustive enumeration grows exponentially in r");

    const auto n = (std::uint64_t{1} << static_cast<unsigned>(r)) - 1ULL;

    Matrix H(
        static_cast<std::size_t>(r)
      , Word(static_cast<std::size_t>(n), Bit{0})
    );

    for (std::uint64_t column = 0; column < n; ++column)
    {
        const auto value = column + 1ULL;

        for (int bit = 0; bit < r; ++bit)
        {
            H[static_cast<std::size_t>(r - 1 - bit)]
             [static_cast<std::size_t>(column)] =
                static_cast<Bit>((value >> static_cast<unsigned>(bit)) &
1ULL);
```

```
        }
    }

    return H;
}

auto extract_column(const Matrix& matrix, const int column_index) -> Word
{
    Word result(static_cast<std::size_t>(row_count(matrix)), Bit{0});

    for (int row = 0; row < row_count(matrix); ++row)
    {
        result[static_cast<std::size_t>(row)] =
            matrix[static_cast<std::size_t>(row)]
                  [static_cast<std::size_t>(column_index)];
    }

    return result;
}

auto gf2_rank(Matrix matrix) -> int
{
    const int rows = row_count(matrix);
    const int cols = column_count(matrix);

    int pivot_row = 0;

    for (int col = 0; col < cols && pivot_row < rows; ++col)
    {
        int selected = -1;

        for (int row = pivot_row; row < rows; ++row)
        {
            if (matrix[static_cast<std::size_t>(row)]
                      [static_cast<std::size_t>(col)] != 0)
            {
                selected = row;
                break;
            }
        }

        if (selected == -1)
        {
            continue;
        }
```

```
        std::swap(
            matrix[static_cast<std::size_t>(pivot_row)]
          , matrix[static_cast<std::size_t>(selected)]
        );

        for (int row = 0; row < rows; ++row)
        {
            if (row == pivot_row)
            {
                continue;
            }

            if (matrix[static_cast<std::size_t>(row)]
                      [static_cast<std::size_t>(col)] == 0)
            {
                continue;
            }

            for (int j = col; j < cols; ++j)
            {
                matrix[static_cast<std::size_t>(row)]
                      [static_cast<std::size_t>(j)] ^=
                    matrix[static_cast<std::size_t>(pivot_row)]
                          [static_cast<std::size_t>(j)];
            }
        }

        ++pivot_row;
    }

    return pivot_row;
}

auto all_columns_nonzero_and_unique(const Matrix& matrix) -> bool
{
    std::set<std::string> seen;

    for (int column = 0; column < column_count(matrix); ++column)
    {
        const auto current = extract_column(matrix, column);

        const bool is_zero =
            std::all_of(
                current.begin()
              , current.end()
              , [](const Bit bit) { return bit == 0; }
```

```
            );

        if (is_zero)
        {
            return false;
        }

        seen.insert(to_bit_string(current));
    }

    return static_cast<int>(seen.size()) == column_count(matrix);
}

auto integer_to_word(const std::uint64_t value, const int width) -> Word
{
    Word word(static_cast<std::size_t>(width), Bit{0});

    for (int bit = 0; bit < width; ++bit)
    {
        word[static_cast<std::size_t>(width - 1 - bit)] =
            static_cast<Bit>((value >> static_cast<unsigned>(bit)) & 1ULL);
    }

    return word;
}

auto enumerate_binary_words(const int width) -> std::vector<Word>
{
    assert((width >= 0) && "word width must be non-negative");
    assert((width < 63) && "word width must fit into uint64_t enumeration");

    const auto count = std::uint64_t{1} << static_cast<unsigned>(width);

    std::vector<Word> words;
    words.reserve(static_cast<std::size_t>(count));

    for (std::uint64_t value = 0; value < count; ++value)
    {
        words.push_back(integer_to_word(value, width));
    }

    return words;
}

auto encode_binary_linear(const Word& message, const Matrix& generator) ->
Word
```

```
{
    assert(
        (static_cast<int>(message.size()) == row_count(generator)) &&
        "message size must equal generator row count"
    );

    Word codeword(static_cast<std::size_t>(column_count(generator)), Bit{0});

    for (int column = 0; column < column_count(generator); ++column)
    {
        Bit value = 0;

        for (int row = 0; row < row_count(generator); ++row)
        {
            value ^= static_cast<Bit>(
                message[static_cast<std::size_t>(row)] &
                generator[static_cast<std::size_t>(row)]
                         [static_cast<std::size_t>(column)]
            );
        }

        codeword[static_cast<std::size_t>(column)] = value;
    }

    return codeword;
}

auto generate_codewords(const Matrix& generator) -> std::vector<Word>
{
    const auto messages = enumerate_binary_words(row_count(generator));

    std::vector<Word> codewords;
    codewords.reserve(messages.size());

    for (const auto& message : messages)
    {
        codewords.push_back(encode_binary_linear(message, generator));
    }

    return codewords;
}

auto hamming_weight(const Word& word) -> int
{
    return std::accumulate(
        word.begin()
```

```
      , word.end()
      , 0
      , [](const int acc, const Bit bit)
        {
            return acc + static_cast<int>(bit);
        }
    );
}

auto hamming_distance(const Word& lhs, const Word& rhs) -> int
{
    assert((lhs.size() == rhs.size()) && "words must have the same length");

    int distance = 0;

    for (std::size_t i = 0; i < lhs.size(); ++i)
    {
        distance += static_cast<int>(lhs[i] != rhs[i]);
    }

    return distance;
}

auto bpsk_correlation(const Word& lhs, const Word& rhs) -> int
{
    assert((lhs.size() == rhs.size()) && "words must have the same length");

    int correlation = 0;

    for (std::size_t i = 0; i < lhs.size(); ++i)
    {
        const int a = lhs[i] == 0 ? 1 : -1;
        const int b = rhs[i] == 0 ? 1 : -1;

        correlation += a * b;
    }

    return correlation;
}

auto verify_simplex_dual_of_hamming(const int r) -> VerificationReport
{
    const auto H = build_hamming_parity_check_matrix(r);
    const auto codewords = generate_codewords(H);

    const int n = column_count(H);
```

```
    const int k = row_count(H);
    const int expected_weight = 1 << (r - 1);

    std::set<int> observed_weights;
    int nonzero_count = 0;

    for (const auto& codeword : codewords)
    {
        const int weight = hamming_weight(codeword);

        if (weight == 0)
        {
            continue;
        }

        observed_weights.insert(weight);
        ++nonzero_count;
    }

    std::set<int> observed_distances;
    std::set<int> observed_bpsk_correlations;

    for (std::size_t i = 0; i < codewords.size(); ++i)
    {
        for (std::size_t j = i + 1; j < codewords.size(); ++j)
        {
            observed_distances.insert(hamming_distance(codewords[i],
codewords[j]));
            observed_bpsk_correlations.insert(bpsk_correlation(codewords[i],
codewords[j]));
        }
    }

    return VerificationReport{
        .r = r
      , .n = n
      , .k = k
      , .expected_weight = expected_weight
      , .nonzero_codeword_count = nonzero_count
      , .generator_has_full_row_rank = (gf2_rank(H) == k)
      , .all_columns_nonzero_and_unique = all_columns_nonzero_and_unique(H)
      , .all_nonzero_weights_equal =
            (observed_weights.size() == 1) &&
            (*observed_weights.begin() == expected_weight)
      , .all_pairwise_distances_equal =
            (observed_distances.size() == 1) &&
```

```
            (*observed_distances.begin() == expected_weight)
      , .all_bpsk_correlations_equal =
            (observed_bpsk_correlations.size() == 1) &&
            (*observed_bpsk_correlations.begin() == -1)
      , .observed_weights = std::move(observed_weights)
      , .observed_distances = std::move(observed_distances)
      , .observed_bpsk_correlations = std::move(observed_bpsk_correlations)
    };
}

auto print_set(const std::set<int>& values) -> void
{
    std::cout << "{";

    bool first = true;

    for (const int value : values)
    {
        if (!first)
        {
            std::cout << ", ";
        }

        std::cout << value;
        first = false;
    }

    std::cout << "}";
}

auto print_report(const VerificationReport& report) -> void
{
    std::cout
        << "r=" << report.r
        << ", parameters=[" << report.n << ", " << report.k << "]\n";

    std::cout
        << "  Full row rank: "
        << (report.generator_has_full_row_rank ? "true" : "false")
        << "\n";

    std::cout
        << "  Nonzero unique columns: "
        << (report.all_columns_nonzero_and_unique ? "true" : "false")
        << "\n";
```

```
    std::cout
        << "  Expected nonzero weight: "
        << report.expected_weight
        << "\n";

    std::cout << "  Observed nonzero weights: ";
    print_set(report.observed_weights);
    std::cout << "\n";

    std::cout << "  Observed pairwise distances: ";
    print_set(report.observed_distances);
    std::cout << "\n";

    std::cout << "  Observed BPSK correlations: ";
    print_set(report.observed_bpsk_correlations);
    std::cout << "\n";

    std::cout
        << "  Simplex by weights: "
        << (report.all_nonzero_weights_equal ? "true" : "false")
        << "\n";

    std::cout
        << "  Simplex by distances: "
        << (report.all_pairwise_distances_equal ? "true" : "false")
        << "\n";

    std::cout
        << "  Equidistant BPSK signal set: "
        << (report.all_bpsk_correlations_equal ? "true" : "false")
        << "\n\n";
}

int main()
{
    for (const int r : {2, 3, 4, 5})
    {
        print_report(verify_simplex_dual_of_hamming(r));
    }

    return 0;
}
```

Результат выполнения:

```
r=2, parameters=[3, 2]
  Full row rank: true
  Nonzero unique columns: true
  Expected nonzero weight: 2
  Observed nonzero weights: {2}
  Observed pairwise distances: {2}
  Observed BPSK correlations: {-1}
  Simplex by weights: true
  Simplex by distances: true
  Equidistant BPSK signal set: true

r=3, parameters=[7, 3]
  Full row rank: true
  Nonzero unique columns: true
  Expected nonzero weight: 4
  Observed nonzero weights: {4}
  Observed pairwise distances: {4}
  Observed BPSK correlations: {-1}
  Simplex by weights: true
  Simplex by distances: true
  Equidistant BPSK signal set: true

r=4, parameters=[15, 4]
  Full row rank: true
  Nonzero unique columns: true
  Expected nonzero weight: 8
  Observed nonzero weights: {8}
  Observed pairwise distances: {8}
  Observed BPSK correlations: {-1}
  Simplex by weights: true
  Simplex by distances: true
  Equidistant BPSK signal set: true

r=5, parameters=[31, 5]
  Full row rank: true
  Nonzero unique columns: true
  Expected nonzero weight: 16
  Observed nonzero weights: {16}
  Observed pairwise distances: {16}
  Observed BPSK correlations: {-1}
  Simplex by weights: true
  Simplex by distances: true
  Equidistant BPSK signal set: true
```